

AL MANARA BANK

Java Programming — Final OOP Assignment

Detailed Student Specification

Deliverable: A working Java console application and all source files.

Important: This document describes the banking problem and the behavior your program must provide. It does not tell you exactly how to design your classes or which OOP technique to use. You are expected to analyze the requirements and choose an appropriate design.

1. Assignment Goal

Al Manara Bank currently manages customer and account information manually. This causes mistakes when employees handle different account types because each type has different rules. Your task is to build a console-based Java application that helps a branch employee register customers, open accounts, perform transactions, search records, and close accounts.

The program should behave like a small banking system. It must protect the data from invalid operations and should clearly explain to the employee why an operation is rejected.

2. What You Are Expected To Demonstrate

Your solution should demonstrate that you can take a real-world problem and turn it into a working Java program. The solution should make sensible use of Java basics, arrays, methods, classes and objects, constructors, data protection, and other OOP features where they are useful. You are responsible for deciding how the design should be organized.

3. General Rules

- Use Java only. The application must run from the console.
- Do not use a database, GUI, Spring, or external libraries.
- Store the system data in memory using arrays. You may choose the array structure that makes sense for your design.
- Do not ask the user to manually type customer IDs or account numbers when the system is supposed to generate them.
- Invalid input should not crash the program.
- Keep the program organized into logical classes and methods. Avoid putting the entire application inside `main()`.

4. The Banking Scenario

Al Manara Bank has one branch. The branch serves many customers, and a customer may have more than one account. The bank currently offers three account types: Savings, Current, and Fixed Deposit.

All accounts have common information such as an account number, an owner, a balance, and a status. However, the rules for transactions are not identical. For example, a savings account cannot become negative, while a current account may use an overdraft. A fixed deposit is locked until its maturity period.

The system must therefore make sure that the correct rules are applied every time an employee performs an operation. The employee should not have to remember special rules manually.

5. Customers

A customer represents a person who uses the bank. When registering a customer, the employee enters:

- Full name
- National ID
- Phone number (optional)
- Customer tier

The system generates a unique customer ID. The customer ID must never be typed manually by the employee.

Customer tiers

Each customer belongs to one of three tiers:

- **STANDARD:** regular customer.
- **SILVER:** customer with a different monthly fee and interest bonus.
- **GOLD:** customer with a different monthly fee and interest bonus.

The exact fee and bonus values are part of your system configuration. Choose reasonable fixed values and document them clearly in your program.

6. Account Status

Every account has one of the following states:

- **ACTIVE** — normal transactions are allowed.
- **FROZEN** — transactions are not allowed.
- **CLOSED** — the account has been closed and cannot be used for normal transactions.

A transaction attempted on a frozen or closed account must be rejected with a clear message.

7. Account Types

7.1 Savings Account

A savings account stores money for normal saving and earns annual interest.

- The balance must never become negative.
- A withdrawal is allowed only when enough money is available.
- The system must count successful withdrawals made during the current month.
- The account has an annual interest rate.

7.2 Current Account

A current account is allowed to use an overdraft up to a predefined limit.

- The balance may become negative.
- The balance must never go below the negative overdraft limit.
- Example: if the overdraft limit is \$1,000, a balance of \$200 may be reduced to -\$800, but a withdrawal that would produce -\$801 must be rejected.
- The system should indicate whether the account is currently using its overdraft.

7.3 Fixed Deposit Account

A fixed deposit is money placed in the bank for a specified number of months.

- The account has an interest rate.
- The account has a duration in months.
- The system tracks how many months have passed.
- Money cannot be withdrawn before the account reaches maturity.
- If a withdrawal is attempted too early, the program must reject it and show how many months remain.
- After the maturity period is reached, withdrawal is allowed according to the account's rules.

8. Common Account Information

Regardless of type, the system needs enough information to identify an account, its owner, its current balance, its status, and how many successful transactions have been performed.

The account number must be generated automatically and must be unique within the application.

9. Main Menu

When the program starts, display a menu similar to the following:

Option	Operation
1	Register New Customer
2	Open New Account
3	Deposit Money
4	Withdraw Money
5	Transfer Between Accounts
6	Display Customer Accounts
7	Display All Branch Accounts
8	Search Account by Number
9	Search Accounts by Type
10	Close an Account
0	Exit

10. Menu Option 1 — Register New Customer

Ask the employee for the customer's full name, national ID, optional phone number, and tier.

Before creating the customer, validate the information. The name cannot be empty. The national ID must not already belong to another customer. If a phone number is entered, it must contain only digits and must be between 7 and 15 digits.

When registration succeeds, generate a customer ID automatically and display the new customer's ID and details.

```
Example: if the generated ID is 1001, the program may display: Customer registered successfully.  
Customer ID: 1001.
```

11. Menu Option 2 — Open New Account

Ask for the customer ID. If it does not exist, do not create an account.

Then ask the employee to select an account type and enter the required information for that type. Check the minimum opening balance configured for the selected type. If the amount is invalid, the account must not be created.

The system generates the account number automatically, creates the account record, associates it with the customer, stores it in the account storage, and updates the customer's account count.

12. Menu Option 3 — Deposit Money

Ask for an account number and a deposit amount.

Find the account. The account must exist and must allow transactions. The amount must be positive and should satisfy the minimum transaction amount chosen for the system.

If successful, increase the account balance, increase its successful transaction count, and display the new balance.

Example: Current balance = \$500. Deposit = \$200. New balance = \$700.

13. Menu Option 4 — Withdraw Money

Ask for an account number and withdrawal amount. Find the account and validate its status and the amount.

The withdrawal must then follow the rules of the particular account type:

- **Savings:** reject the operation if the resulting balance would be below zero; on success, increase the monthly withdrawal counter.
- **Current:** allow the balance to become negative, but never below the allowed overdraft limit.
- **Fixed Deposit:** reject the operation if the account has not matured. If it has matured, allow the withdrawal according to the account rules.

Every successful withdrawal increases the transaction count. Every rejected withdrawal must leave the balance unchanged.

14. Menu Option 5 — Transfer Between Accounts

Ask for a source account number, destination account number, and amount.

Both accounts must exist and must be different. The source must be able to complete the withdrawal. The destination must be able to receive the deposit.

A transfer is one complete operation. If the source withdrawal succeeds but the destination deposit cannot be completed, the money must be restored to the source so that the failed transfer does not lose funds.

Example: Source has \$1,000. Transfer \$250. If successful, source becomes \$750 and destination increases by \$250.

Display a clear success or failure message.

15. Menu Option 6 — Display Customer Accounts

Ask for a customer ID. If the customer exists, display the customer's information and all accounts belonging to that customer.

For each account, display at least:

- Account number
- Account type
- Balance
- Status
- Transaction count

At the end, display the combined balance of all accounts owned by that customer.

16. Menu Option 7 — Display All Branch Accounts

Display all accounts currently stored by the branch. A useful display should include account number, owner name, type, balance, status, and transaction count.

If there are no accounts, display a suitable message instead of showing an empty or confusing table.

17. Menu Option 8 — Search Account by Number

Ask for an account number and search the stored accounts.

The search should be case-insensitive if your account-number format contains letters. If found, display the complete information relevant to that account, including information specific to its type.

For example, a current account can display its overdraft limit, while a fixed deposit can display its duration, elapsed months, and whether it has matured.

18. Menu Option 9 — Search Accounts by Type

Ask the employee to choose an account type. Display all accounts of that type.

At the end, display the number of matching accounts and the combined balance of those accounts.

19. Menu Option 10 — Close an Account

An account cannot be closed simply because the employee selects the close option.

Before closing, verify that:

- The account exists.
- The account is not already closed.
- The balance is exactly \$0.
- A fixed deposit is not still locked before maturity.

If all conditions are satisfied, mark the account as closed and update the customer's number of open accounts.

20. Menu Option 0 — Exit

The program should stop its menu loop and display a short farewell message.

21. Input Validation

The application should handle invalid input without crashing. At minimum, consider the following situations:

- Selecting a menu number that does not exist.
- Entering text where a number is expected.
- Entering a negative amount or zero when a positive amount is required.
- Using an account number that does not exist.
- Using a customer ID that does not exist.
- Registering a duplicate national ID.
- Trying to transact on a frozen or closed account.
- Trying to withdraw more than the allowed amount.
- Trying to close an account with a non-zero balance.
- Trying to withdraw from an unmatured fixed deposit.

22. Data Storage Requirements

All information must remain available while the program is running. Use arrays as the main in-memory storage mechanism.

You will need to decide how to organize the information. For example, you may need a collection of customers and another collection of accounts. You may use one-dimensional arrays, arrays of objects, or another array-based arrangement that fits your design.

The application should support a reasonable maximum capacity. Before adding a new customer or account, check whether the storage has room. If it is full, display an appropriate message rather than causing an exception.

23. Automatic IDs

Customer IDs and account numbers must be generated by the application. They must not be entered manually by the employee.

Use internal counters or another simple mechanism to ensure that newly created records receive unique identifiers during the program's execution.

24. Relationships Between Records

A customer can own multiple accounts. Each account belongs to one customer. When displaying a customer's accounts, the system must find the accounts that belong to that customer.

Think carefully about what information an account needs in order to identify its owner. The system should not depend on the employee repeatedly typing the owner's name to connect records.

25. Program Organization

Your program should be divided into logical responsibilities. The main menu should mainly coordinate user interaction rather than containing every business rule in one large block of code.

Use methods for repeated operations and for tasks that have a clear responsibility. Keep validation, searching, calculations, and display operations organized so that the program remains readable.

26. Design Challenge

The requirements intentionally describe several account types that share information but behave differently. You must decide how to represent this in Java so that the design is easy to maintain and another account type could be added later.

Do not duplicate large amounts of identical logic simply because two account types share the same behavior. At the same time, do not force completely different rules into one confusing method. Find a clean design based on the requirements.

27. Recommended Testing Scenarios

Before submission, test your program with realistic scenarios. At minimum, test the following:

Test	Expected result
Customer registration	Valid customer is created and receives an automatic ID.
Duplicate national ID	Second registration is rejected.
Savings withdrawal	Withdrawal that would make balance negative is rejected.
Current overdraft	Withdrawal is accepted within the overdraft limit and rejected beyond it.
Fixed deposit early withdrawal	Withdrawal is rejected and remaining months are displayed.
Transfer	Successful transfer decreases source and increases destination.
Failed transfer	Funds are not lost if the transfer cannot complete.
Account closure	Non-zero balance prevents closure.
Invalid input	Program continues running and asks for valid input.
Search	Correct account/customer information is displayed.

28. Submission Checklist

- The application starts successfully and shows a menu.
- Customers can be registered and searched.
- Customer IDs are generated automatically.
- Accounts can be opened for existing customers.
- Account numbers are generated automatically.
- Deposits and withdrawals follow the correct rules.
- Transfers work correctly and do not lose money when they fail.
- Customer and account searches work.
- Accounts can only be closed when the required conditions are satisfied.
- Invalid input does not terminate the program unexpectedly.
- The source code is organized and readable.
- All required source files are submitted.

29. Final Design Challenge

This assignment intentionally does not provide a class diagram or tell you exactly which class should inherit from which class. You must analyze the requirements and decide how to organize your program.

There are several account types with shared information and different behaviors. There are also customers that can own multiple accounts. Think about how you can represent these relationships cleanly while keeping the program easy to understand and modify.

You should be able to explain your design if asked: why you created each class, what information belongs to each object, where the business rules are handled, and how the different parts of the program communicate.

30. What Will Be Evaluated

Your program will be evaluated based on both correctness and design quality.

- **Functionality:** Does the program perform the required operations correctly?
- **Business rules:** Are Savings, Current, and Fixed Deposit rules correctly enforced?
- **Data handling:** Are customers and accounts stored and found correctly using arrays?
- **Input handling:** Does the program handle invalid input without crashing?
- **Code organization:** Is the program divided into sensible classes and methods?
- **OOP design:** Did you make appropriate design decisions based on the problem rather than putting everything into one class?
- **Readability:** Is the code understandable and reasonably clean?

31. Important Note

There is no single required class diagram or one required implementation. You are being evaluated on whether your program correctly solves the banking problem and whether your design is sensible, organized, and maintainable.

Read the requirements carefully before coding. First understand the rules, then plan your data and operations, and finally implement and test the application.

Good luck!